

GOVERNMENT ARTS AND SCIENCE COLLEGE

IDAPPADI - 637 102, SALEM DISTRICT, TAMIL NADU

DEPARTMENT OF COMPUTER SCIENCE & PHYSICAL EDUCATION



SMART SPORTS MANAGEMENT SYSTEM

COMPLETE SOURCE CODE ARCHITECTURE, FOLDER DIRECTORY & MODULE
EXPLANATION GUIDE

Project Specification & Code Blueprint Manual

This technical documentation manual provides a comprehensive, file-by-file, module-by-module breakdown of the GASC Idappadi Smart Sports Management System. It explains the purpose of every folder, frontend UI/UX templates, JavaScript controllers, REST API routes, Express middleware, Supabase Cloud PostgreSQL schema, utility algorithms, security protocols, and operational workflows.

System Core	Node.js (Express.js) + Modern Vanilla JS + CSS3 + Supabase
Database Engine	Supabase PostgreSQL (Cloud Hosted, 16 Relational Tables, RLS, Triggers)
Target Institution	Government Arts and Science College, Idappadi (GASC)
User Roles	1. Physical Director / Admin 2. Student Athletes 3. Public
Document Version	v2.5.0 Production Ready (Full Technical Reference)

Prepared for Project Review, Viva-Voce Examination & Institutional Deployment

1. System Overview & Technology Stack

The **GASC Idappadi Smart Sports Management System** is a full-stack, enterprise-grade sports management platform designed specifically for Government Arts and Science College, Idappadi. The application automates student sports enrollment, master roster verification, biometric-ready daily attendance tracking, sports equipment inventory & fine calculations, inter-collegiate competition management, merit achievement awards, and AI-driven performance analytics.

Layer	Technologies Used	Key Role & Responsibility
Frontend UI/UX	HTML5, Vanilla CSS3 (Custom Responsive Design System), Modern Vanilla JavaScript (ES6+), FontAwesome Icons, Chart.js	Responsive portal for Public, Admin Dashboard, and Student Athlete Portal without heavy third-party framework overhead.
Backend Server	Node.js, Express.js REST APIs, CORS, Multer, Bcrypt.js, JSON Web Tokens (JWT), Nodemailer	Handles authentication, business logic, file uploads, OTP generation/verification, email notifications, and automated report generation.
Database & Cloud	Supabase Cloud PostgreSQL (@supabase/supabase-js), SQL Views, Triggers, Functions, Foreign Key Cascades	Cloud relational database managing 16 relational tables with ACID compliance, instant real-time data access, and automated seeders.
AI & Analytics	Heuristic AI Rule Engine (JS Service), Chart.js Data Visualizations	Generates predictive sports performance insights, low-stock forecasts, attendance drop alerts, and department medal summaries.

High-Level Architectural Workflow

The system follows a standard **3-Tier MVC-style Architecture** (Client View → REST Controller/Service Layer → Supabase Database Layer):

- **1. Client Layer (Browser):** Public visitors access institutional news, achievements, and events. Students log into their dedicated Athlete Dashboard. Physical Directors manage complete institutional sports via the secured Admin Dashboard.
- **2. Application Gateway (Express.js):** Receives HTTP requests, validates JWT authorization tokens, sanitizes form inputs, checks role permissions (admin vs student), handles file multipart streams, and routes payloads to respective controllers.
- **3. Service & Utility Engines:** Performs OTP email dispatch (Nodemailer), AI performance scoring, inventory threshold calculations, and Excel roster parsing.
- **4. Supabase PostgreSQL Storage:** Executes relational queries, enforces foreign keys, updates stock quantities via atomic updates, and maintains permanent audit logs.

2. Complete Folder Directory Structure

The repository is organized into a clean, modular structure dividing client-side static assets and server-side business logic:

Path / Directory	Contents & Work Handled
/ (Root)	Configuration files (package.json, .env), SQL schema (supabase_schema.sql), import scripts (import-students.js), and startup scripts (start.bat, run.ps1).
/client/public/	Static frontend web application. Contains all HTML pages (Public, Student, Admin), CSS stylesheets, JavaScript files, college logos, images, and user upload directory.
/client/public/css/	Contains style.css (Global design system, CSS variables, typography, responsive grids, dark/light cards, modal styles, and animations).

/client/public/js/	Client-side controllers: api.js (Fetch wrapper), auth.js (Auth & OTP), admin.js (Admin Dashboard logic), student.js (Athlete portal), public.js (Website interactions), and charts.js (Analytics visualizers).
/client/public/uploads/	Server upload destination for player profile avatars, competition posters, and achievement certificate attachments.
/server/	Node.js Express backend root. Contains server.js (main entry point) and subdirectories for config, controllers, routes, middleware, models, services, and utils.
/server/config/	Configuration modules: supabase.js (Supabase Cloud client connector) and db.js (Database status checker).
/server/controllers/	15 Controller modules containing complete business logic for Authentication, Players, Sports, Equipment, Competitions, Teams, Achievements, Attendance, Practice, Roster, Analytics, etc.
/server/middleware/	Express middlewares: authMiddleware.js (JWT verification, role authorization) and uploadMiddleware.js (Multer file storage and type filters).
/server/models/	Schema reference definitions and ORM data structures for all 16 database entities.
/server/routes/	16 Express router files mapping HTTP endpoints (GET, POST, PUT, DELETE) to corresponding controller handler functions.
/server/services/	Specialized business services: aiInsightsService.js, notificationService.js, stockService.js, and supabaseService.js.
/server/utils/	Utility modules: emailService.js (SMTP emails), otpStore.js (In-memory OTP storage), seedSupabase.js (Auto-seeder), supabaseHelper.js, and supabaseSync.js.

3. Frontend UI & JavaScript Modules

3.1 Frontend HTML Pages Breakdown

The frontend is built as a lightweight, lightning-fast Single/Multi-Page Application hybrid utilizing standard HTML5 and responsive modern styling:

HTML File	Target Users	Functionality & Code Features
index.html	Public, All Visitors	College Sports Portal Landing Page. Displays college hero banner, Quick Statistics (Athletes, Sports, Medals), Upcoming Tournaments, Recent Achievements, Sports Offered showcase, Gallery preview, and Contact section.
login.html	Students & Admin	Dual Login Portal. Supports Student Login (using College Register Number / Roll No + Password) and Admin / Physical Director login. Includes 'Forgot Password' OTP flow.
register.html	Enrolled Students	Smart Student Athlete Registration. Enforces College Roster Verification (matches Register Number + DOB with college master records), sends Email OTP, and creates player account.
admin-dashboard.html	Physical Director / Admin	Comprehensive Command Center. Contains 12 full management tabs: Live Analytics & AI Insights, Student Athletes, Sports Master, Equipment Inventory, Competitions, Teams, Achievements, Attendance Register, Practice Sessions, College Master Roster, Gallery, and System Settings.
student-dashboard.html	Logged-in Students	Athlete Portal. Features Student Profile Card, Sports Enrolled, Daily Attendance History & Percentage, Equipment Borrowed & Return Status, Competition Registrations, and Achievement Submissions.
achievements.html	Public / Students	Hall of Fame & Trophy Wall. Showcases Zonal, State, Inter-Collegiate, and National medals won by GASC athletes with filter by sport and year.
competitions.html	Public / Students	College & University Fixtures. Lists Upcoming, Ongoing, and Completed tournaments, eligibility rules, venue details, and registration links.
sports.html & gallery.html	Public / Visitors	Explores sports infrastructure (Athletics track, Volleyball court, Kabaddi ground, Indoor badminton/chess) and photo galleries.
about.html & contact.html	Public / Visitors	Institutional profile, Physical Director message, college vision/mission, sports department contact details, and location map.

3.2 Frontend JavaScript Architecture

The client-side logic is organized into specialized, modular JS files located in client/public/js/:

JavaScript File	Key Functions & Internal Logic
api.js (API Gateway)	• <code>apiFetch(endpoint, options)</code>: Centralized HTTP wrapper that attaches JWT Bearer tokens to headers and standardizes error responses. • <code>uploadFile(endpoint, formData)</code>: Handles multipart form uploads for profile pictures, certificate proofs, and tournament posters. • <code>showToast(message, type)</code>: Displays animated success, warning, and error toast alerts.
auth.js (Authentication)	• <code>login(credentials)</code>: Submits student/admin login and stores JWT token & user session in <code>localStorage</code>. • <code>verifyRosterAndSendOtp()</code>: Validates register number against college roster before triggering 6-digit email OTP. • <code>verifyOtpAndRegister()</code>: Confirms OTP and finalizes athlete profile creation. • <code>forgotPasswordFlow()</code> & <code>logout()</code>: Handles password recovery and secure session termination.

admin.js (Admin Controller)	• loadDashboardStats(): Fetches counts, charts, and AI predictive insights. • renderPlayers() / exportPlayersCSV(): Manages athlete table, filters by sport/dept, and downloads data. • manageEquipment(): Handles stock addition, item issuance, return verification, damage logs, and fine assessments. • markAttendance(): Fast daily attendance grid for marking Present/Absent/OD. • manageCompetitions() & manageRoster(): CRUD for tournaments and Excel bulk student roster upload.
student.js (Student Portal)	• loadStudentProfile(): Fetches athlete metrics (height, weight, blood group, department, batch). • renderMyAttendance(): Visualizes attendance percentage bar and OD session history. • renderMyEquipment(): Lists active borrowed gear with return due dates. • registerForEvent(): One-click tournament registration.
charts.js (Visualizer)	• initGenderChart(): Renders Doughnut chart of Male vs Female athlete participation. • initDepartmentChart(): Renders Bar chart of athletes across CS, Maths, Physics, Chemistry, English, Tamil, B.Com. • initMonthlyAttendanceChart(): Renders Line trend of daily training attendance.

4. Backend Server, Controllers & Routes

The backend server is powered by Express.js and structured into RESTful route handlers and modular controller functions:

Controller / Route	HTTP Endpoints	Business Logic & Code Execution
authController.js /api/auth	POST /login POST /register POST /send-otp POST /verify-otp POST /reset-password	- Verifies student credentials and admin passwords using bcrypt.compare. - Cross-checks student register numbers against college_student_roster. - Generates cryptographically secure 6-digit OTPs and delivers via Nodemailer. - Issues signed JSON Web Tokens (JWT) with 7-day expiration.
playerController.js /api/players	GET / GET /profile PUT /profile GET /:id	- Retrieves athlete profile, sports history, and biometric stats. - Handles avatar image upload via Multer and updates Supabase player record. - Provides admin-level filtering by sport, department, batch, and gender.
equipmentController.js /api/equipment	GET / POST / POST /issue POST /return GET /transactions	- Manages sports inventory items (Cricket bats, Volleyballs, Nets, Javelins). - Atomically decrements available stock upon gear issue. - Computes overdue late fines and damage charges on return, automatically incrementing available stock.
competitionController.js /api/competitions	GET / POST / POST /:id/register PUT /:id/results	- Admin creates upcoming tournaments with entry deadlines, venue, and rules. - Students register solo or as team captains. - Admin updates winners, scores, and medal tallies.
attendanceController.js /api/attendance	POST /mark GET /daily GET /summary GET /student/:id	- Bulk marks daily sports training attendance (Present / Absent / OD). - Computes per-student attendance percentage required for academic On-Duty (OD) exam exemptions.
practiceController.js /api/practice	GET / POST / PUT /:id DELETE /:id	Physical Director schedules daily coaching drills, practice matches, and fitness sessions with specific grounds and timing.
achievementController.js /api/achievements	GET / POST / PUT /:id/verify	- Records medals, trophies, and merit certificates (Zonal, State, National). - Students submit achievement claims with certificate proof for admin verification.
teamController.js & sportController.js	GET / POST / PUT on /api/teams and /api/sports	- Manages sports disciplines offered at GASC Idappadi. - Forms official college teams (Cricket, Volleyball, Kabaddi, Kho-Kho, Chess) and assigns captains.
rosterController.js /api/roster	GET / POST /upload-excel POST /verify	- Handles institutional student roster validation. - Supports bulk import of entire college student data from Excel (.xlsx) files to eliminate outsider fake signups.
analyticsController.js /api/analytics	GET /dashboard GET /ai-insights	- Aggregates total athletes, sport popularity, gender ratios, and inventory usage. - Calls aiInsightsService.js to generate heuristic recommendations.

5. Server Middleware, Services & Utilities

5.1 Express Middlewares

Middleware File	Function Name	Detailed Role & Security Operations
authMiddleware.js	authenticateJWT	Extracts the Bearer token from the HTTP Authorization header, verifies its digital signature using JWT_SECRET, and attaches the decoded user payload (req.user) to the request.
authMiddleware.js	requireAdmin	Ensures that the authenticated user possesses the admin role. If a normal student attempts an administrative action, it returns HTTP 403 Forbidden.
uploadMiddleware.js	upload (Multer)	Configures disk storage destination (client/public/uploads/), assigns unique timestamps to filenames, and enforces file type filtering (images: PNG, JPG, JPEG, WEBP; docs: PDF).

5.2 Business Services & System Utilities

Module / Script	Location	Purpose & Implementation Details
aiInsightsService.js	server/services/	AI Heuristic Engine: Analyzes athlete attendance trends (flags <75% attendance), forecasts inventory restock needs (low stock alerts), identifies top-performing college sports departments, and recommends training regimens.
emailService.js	server/utills/	Nodemailer Dispatcher: Sends beautiful HTML emails for Registration OTP verification, Password resets, Tournament confirmation alerts, and Equipment overdue notices via SMTP.
otpStore.js	server/utills/	In-Memory OTP Cache: Stores ephemeral 6-digit registration & password-reset verification tokens with strict 10-minute expiration windows and automatic cleanup.
seedSupabase.js	server/utills/	Automated Database Seeder: Automatically populates Supabase PostgreSQL with default Admin accounts, sample student roster, standard sports (Cricket, Volleyball, Kabaddi), initial equipment stock, and sample competitions.
supabase.js	server/config/	Supabase Client Connector: Initializes the @supabase/supabase-js client using SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY from .env.

6. Database Architecture & Supabase Schema

The system utilizes **Supabase Cloud PostgreSQL** as its relational database. The schema is fully normalized and consists of **16 core tables** linked through foreign key constraints:

Table Name	Primary Key / Foreign Keys	Stored Information & Columns
users	id (UUID / Serial) Unique: register_number, email	User authentication records: register_number, email, password_hash (bcrypt), role ('admin' / 'student'), full_name, is_active, created_at.
college_student_roster	id Unique: register_number	Master college registry: register_number, roll_no, full_name, department, batch_year, gender, date_of_birth, mobile, email, is_registered.
player_profiles	id FK: user_id → users(id)	Athlete biometrics: height_cm, weight_kg, blood_group, emergency_contact, primary_sport_id, secondary_sport_id, t_shirt_size, avatar_url.
sports	id Unique: code	Sports master: name ('Cricket', 'Volleyball', etc.), code, category ('Indoor' / 'Outdoor' / 'Track'), team_size, rules_description, icon.
equipment	id FK: sport_id → sports(id)	Sports inventory: item_name, sport_id, total_quantity, available_quantity, damaged_quantity, low_stock_threshold, storage_room, unit_price.
equipment_transactions	id FK: equipment_id, student_id	Issue/Return logs: equipment_id, student_id, quantity, issued_date, expected_return_date, actual_return_date, status ('Issued' / 'Returned' / 'Overdue'), condition, fine_amount.
competitions	id FK: sport_id → sports(id)	Tournaments: title, sport_id, level ('Zonal' / 'Inter-Collegiate' / 'State' / 'National'), start_date, end_date, venue, registration_deadline, max_participants, status.
competition_registrations	id FK: competition_id, student_id	Student registrations: competition_id, student_id, registration_type ('Solo' / 'Team'), team_name, status ('Pending' / 'Approved' / 'Rejected'), result_position.
attendance	id FK: student_id, sport_id	Daily training attendance: student_id, sport_id, attendance_date, status ('Present' / 'Absent' / 'On-Duty'), marked_by, remarks.
practice_sessions	id FK: sport_id → sports(id)	Coaching schedules: sport_id, session_title, session_date, start_time, end_time, venue_ground, coach_incharge, drill_focus.
achievements	id FK: student_id, sport_id	Merit & Awards: student_id, sport_id, event_title, award_type ('Winner' / 'Runner' / 'Bronze'), medal ('Gold' / 'Silver' / 'Bronze'), certificate_url, verified_by_admin.
teams & team_members	id FK: sport_id, captain_id	College teams: team_name, sport_id, captain_id, academic_year, team members junction table.
notifications & gallery	id	Broadcast notifications, target role/sport, is_read, gallery photo items with captions and categories.
admin_settings & otps	id / Key-Value	Institution metadata, Physical Director name, current academic year, SMTP toggles, and OTP verification records.

7. End-to-End Operational Workflows & Data Flows

7.1 Student Onboarding & Verification Flow

To guarantee that only bona-fide GASC Idappadi students can register, the system implements a strict 3-step verification protocol:

- **Step 1 (Roster Lookup):** The student enters Register Number and Date of Birth on register.html. The frontend queries POST /api/auth/send-otp. The server validates if the student exists in college_student_roster and is not yet registered.
- **Step 2 (OTP Generation & Email):** The backend creates a cryptographically random 6-digit OTP, caches it in otpStore.js with a 10-minute TTL, and dispatches an HTML email to the student's official registered email address via Nodemailer.
- **Step 3 (Account Creation):** The student inputs the received OTP and creates a secure password. The server verifies the OTP, hashes the password using bcrypt.hash(password, 10), creates a row in users, initializes a player_profiles record, marks is_registered = true in the roster, and signs a JWT.

7.2 Sports Equipment Issue & Return Lifecycle

- **Step 1 (Issuance):** The Physical Director selects an item (e.g. Volleyball Net) and inputs the Student's Register Number on admin-dashboard.html.
- **Step 2 (Stock Deduction):** POST /api/equipment/issue creates an equipment_transactions record and decrements available_quantity in the equipment table by 1.
- **Step 3 (Return & Condition Check):** When returning gear, the admin marks condition ('Good' / 'Damaged'). If returned after expected_return_date, an automated late fine is calculated. Stock is incremented back, or moved to damaged_quantity.

7.3 Attendance Tracking & Academic On-Duty (OD) Flow

- **Step 1 (Marking):** Admin opens the Daily Attendance tab, selects the Sport (e.g. Kabaddi), and fast-marks Present / Absent / On-Duty.
- **Step 2 (Aggregation):** The backend stores records in attendance. The system aggregates cumulative attendance percentages per athlete.
- **Step 3 (Exemption Report):** If an athlete has >75% sports training attendance and participates in university zonals, the admin generates an official On-Duty (OD) exemption PDF report for semester exam eligibility.

8. Security Architecture & Viva-Voce Key Points

8.1 Security & Best Practices

- **Password Hashing:** Passwords are never stored in plain text; salted SHA-256 bcrypt hashing with 10 salt rounds is applied.
- **Stateless JWT Authentication:** Tokens are cryptographically signed using HS256 algorithm and validated on every protected API endpoint.
- **Role-Based Access Control (RBAC):** Strict separation of student vs admin capabilities via requireAdmin middleware.
- **SQL Injection & XSS Protection:** Parameterized Supabase client queries prevent SQL injection, and all frontend DOM updates sanitize dynamic user strings.

8.2 Frequently Asked Viva-Voce Questions & Answers

Viva Question	Recommended Technical Answer for Examiner
Q1: Why did you choose Supabase PostgreSQL over MongoDB?	Sports management data is highly relational (e.g., Students link to Profiles, Attendance, Equipment Loans, and Tournaments). PostgreSQL enforces ACID compliance, Foreign Key Cascades, and Referential Integrity, which prevents orphan loan records and inconsistent stock numbers.
Q2: How does the system prevent fake student registrations?	The college admin pre-loads the master student database via Excel. During registration, the student's Register Number and DOB are validated against the roster, followed by 6-digit OTP delivery to the registered email.
Q3: What role does the AI Insights Service play?	It utilizes heuristic statistical models to analyze attendance patterns, predict equipment shortages before major tournaments, and generate department-wise medal conversion metrics.
Q4: How does the equipment stock remain synchronized?	Every equipment transaction executes an atomic database operation: decrementing available stock on issue and incrementing upon verified return.

8.3 How to Run the Project Locally

- **Step 1:** Open project directory in terminal or VS Code.
- **Step 2:** Install dependencies: npm install
- **Step 3:** Configure .env file with SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY.
- **Step 4:** Launch the application: npm run dev or run start.bat
- **Step 5:** Open your web browser at **http://localhost:5000**